PRODUCT REQUIREMENT DOCUMENT (PRD)

Project Title: Rental Listing Operations Manager

Project Type: Application Developer – Product Development (Ops-grade MVP)

Student Name: Anurag Khubalkar

Roll No: 240410700120

Year & Section: 2nd Year, Semester 4

Mentor: Kshitiz Dhooria – Backend Systems, Microservices, Cloud-Native Architecture

Overview

*The **Rental Listing Operations Manager** is an operations-focused backend system designed to manage rental listings and their availability in a reliable, conflict-free, and scalable manner. The project addresses real-world operational challenges such as double bookings, incorrect availability calendars, unclear listing states, and poor communication between customers and operations teams.*

The system enables rental operations teams to create and manage listings, control availability through a structured calendar, apply availability blocks, and prevent overlapping or conflicting bookings using backend-enforced validation logic. A defined listing lifecycle ensures predictable and controlled state transitions.

*Additionally, the project includes a **listing-specific chat system** that allows structured communication between customers and operators regarding availability and booking queries. All critical decisions are handled on the backend to ensure data integrity and ops-grade reliability.*

---

1. Industry + Role Alignment

Industry

- Real Estate Technology
- Rental Marketplaces
- Property Management Platforms

Role Alignment

Application Developer (Product & Operations Systems)

Why This Project Fits the Role

- Solves a real operational problem faced by rental platforms
- Focuses on backend correctness and reliability
- Uses availability logic, validation, and state machines
- Demonstrates system design and scalability thinking
- Goes beyond UI into ops-grade engineering

---

2. Problem Statement

Rental platforms require a reliable system to manage:

- Listing availability
- Booking constraints
- Operational calendar accuracy
- Customer–operator communication

Without a structured backend system:

- Double bookings occur
- Calendar states become inconsistent
- Listing lifecycle becomes unclear
- Communication gaps create operational confusion

This project aims to solve these problems by providing a centralized rental listing operations system with strong backend validation and structured workflows.

---

3. End Users Defined

User Type	Description
Rental Operations Manager	Manages listings and availability

Property Admin	Controls listing states and operational rules
Platform Ops Team	Monitors system health and correctness
Customer Support	Resolves booking and communication issues
Customer / Renter	Communicates regarding availability

---

4. Feature List

4.1 Must-Have Features (Core MVP)

Listing & Setup

- Rental listing creation
- Listing lifecycle states (Draft, Active, Paused, Disabled)
- Listing readiness validator

Availability Management

- Availability calendar management
- Availability block creation
- Partial-day / time-based blocking

Validation & Control

- Date conflict prevention logic
- Backend validation of availability

Data & Operations

- Persistent storage using PostgreSQL
- Operational messaging and alerts

Communication

- Listing-specific chat rooms
- Customer ↔ Operator messaging

- Message history storage
- Backend-validated chat access

4.2 Nice-to-Have (Stretch Goals)

- Bulk availability updates
- Admin dashboard
- Role-based access control (RBAC)
- Audit logs for availability and state changes
- Optimized calendar views (weekly/monthly)
- Notifications for invalid operations

---

5. Tech Stack Selection

Layer	Technology	Reason
Frontend	React	Component-based UI
Backend	Node.js + Express	Fast REST APIs
Database	PostgreSQL	Strong relational integrity
Availability Engine	Custom Logic	Full control over rules
Chat System	REST APIs (WebSocket)	Two-way async communication
Deployment	Cloud	Scalability and availability

---

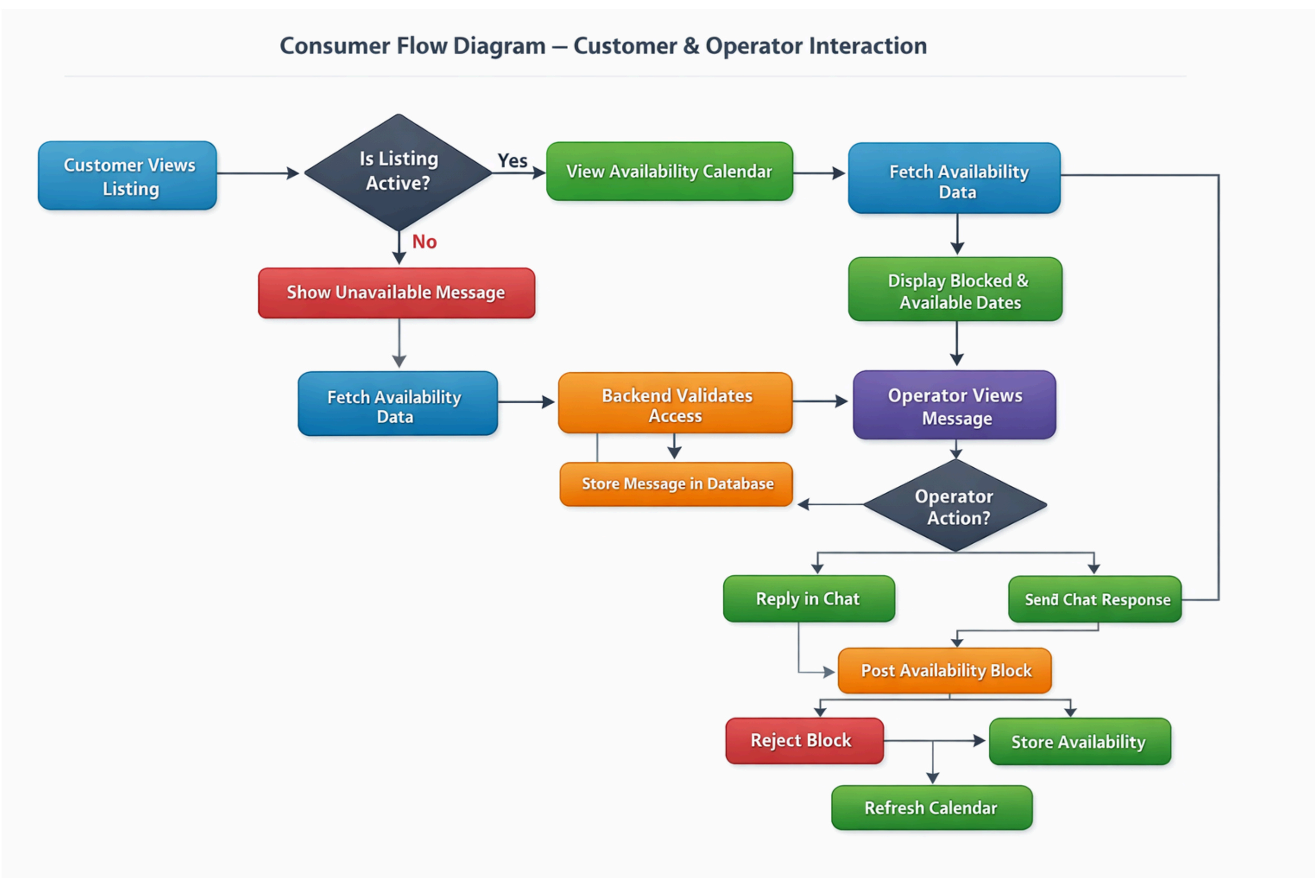
6. Project Proposal

The project proposes building an ops-grade rental listing management system that prioritizes backend correctness over UI complexity. The system is designed to prevent double bookings, maintain consistent availability data, and support structured communication between customers and operations teams.

The scope intentionally avoids over-engineering (e.g., real-time messaging or payment systems) to maintain clarity, correctness, and focus on operational reliability.

---

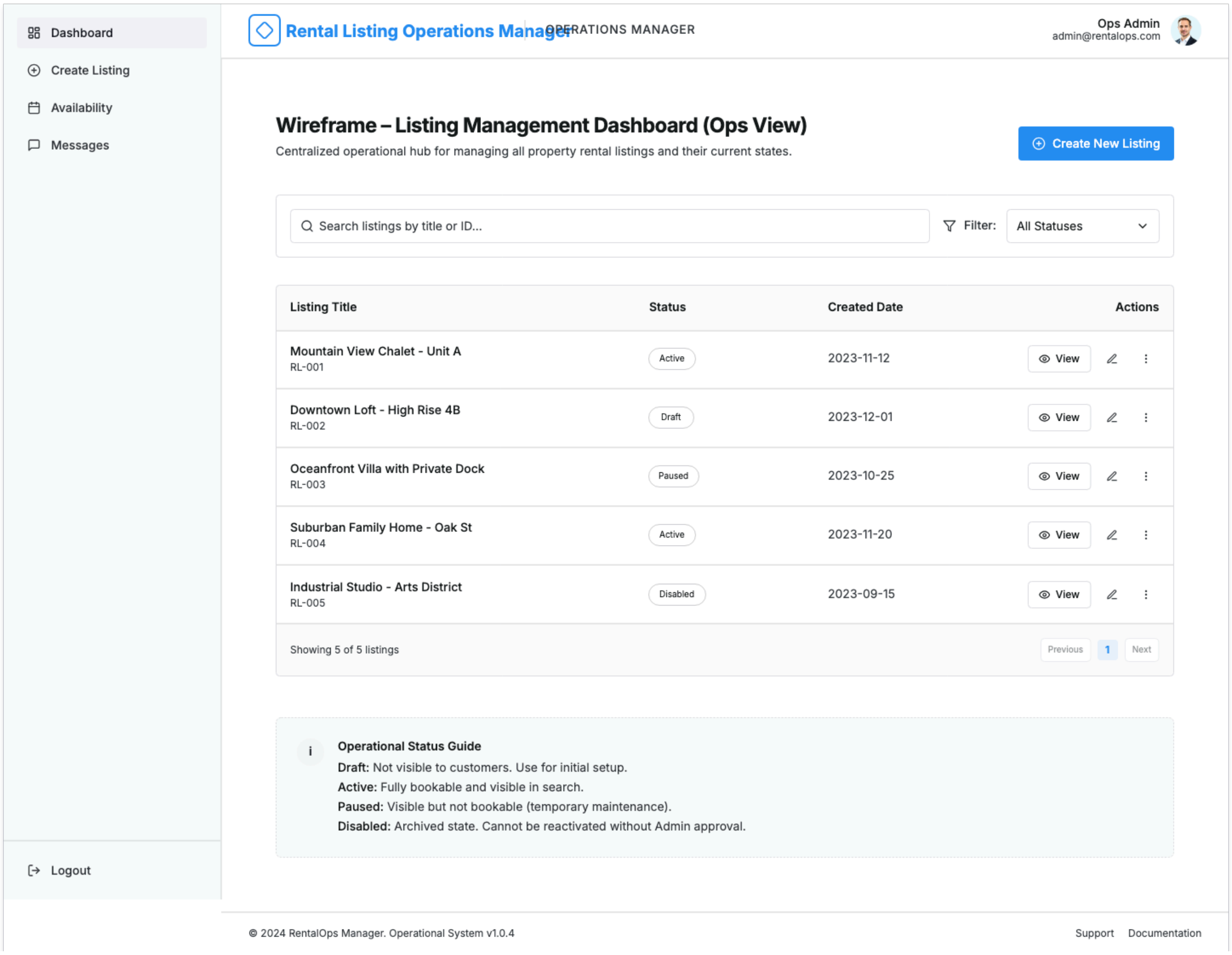
7. Consumer Flow Diagram (User Journey)



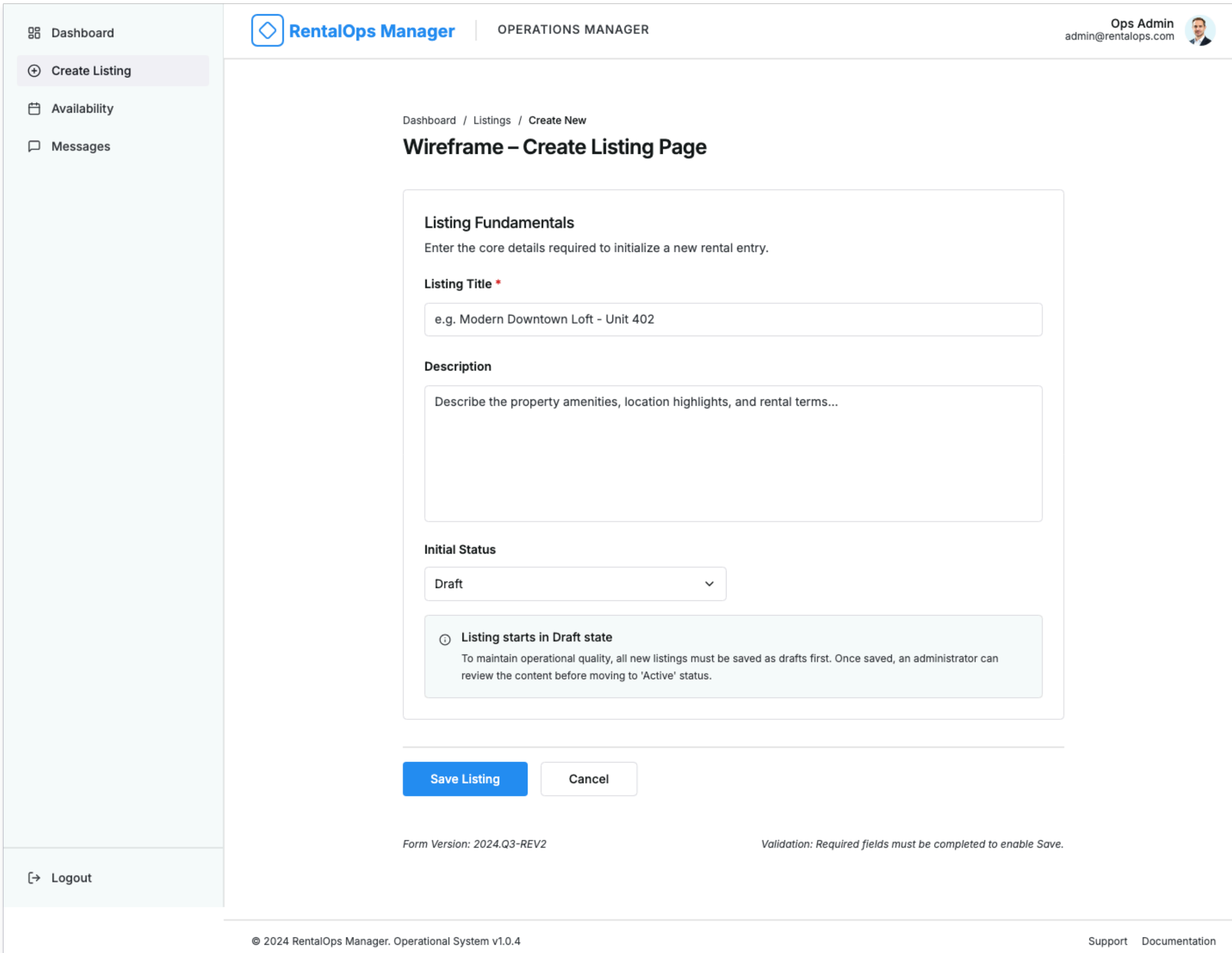
---

8. Wireframes (Low-Fidelity)

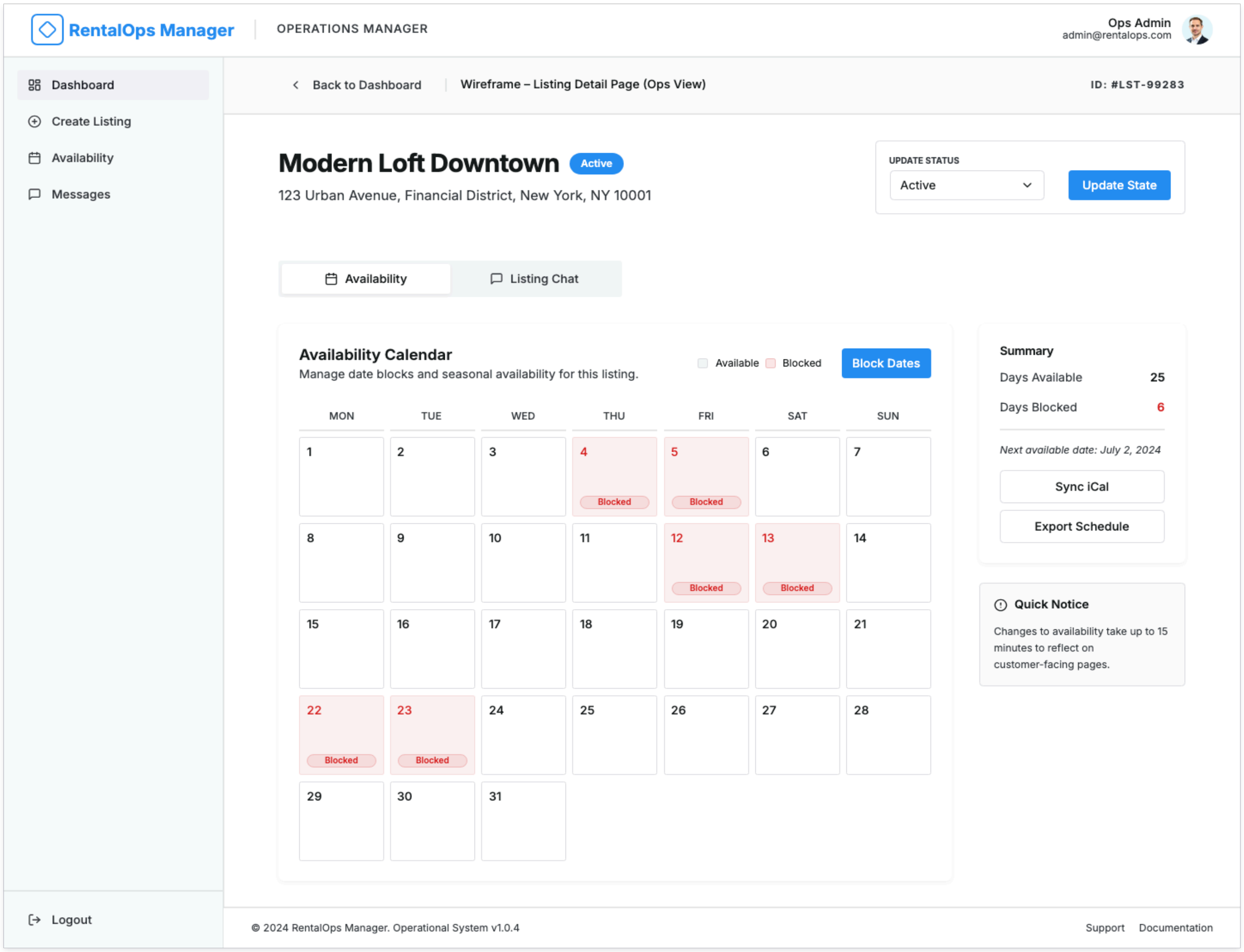
A. Listing Dashboard



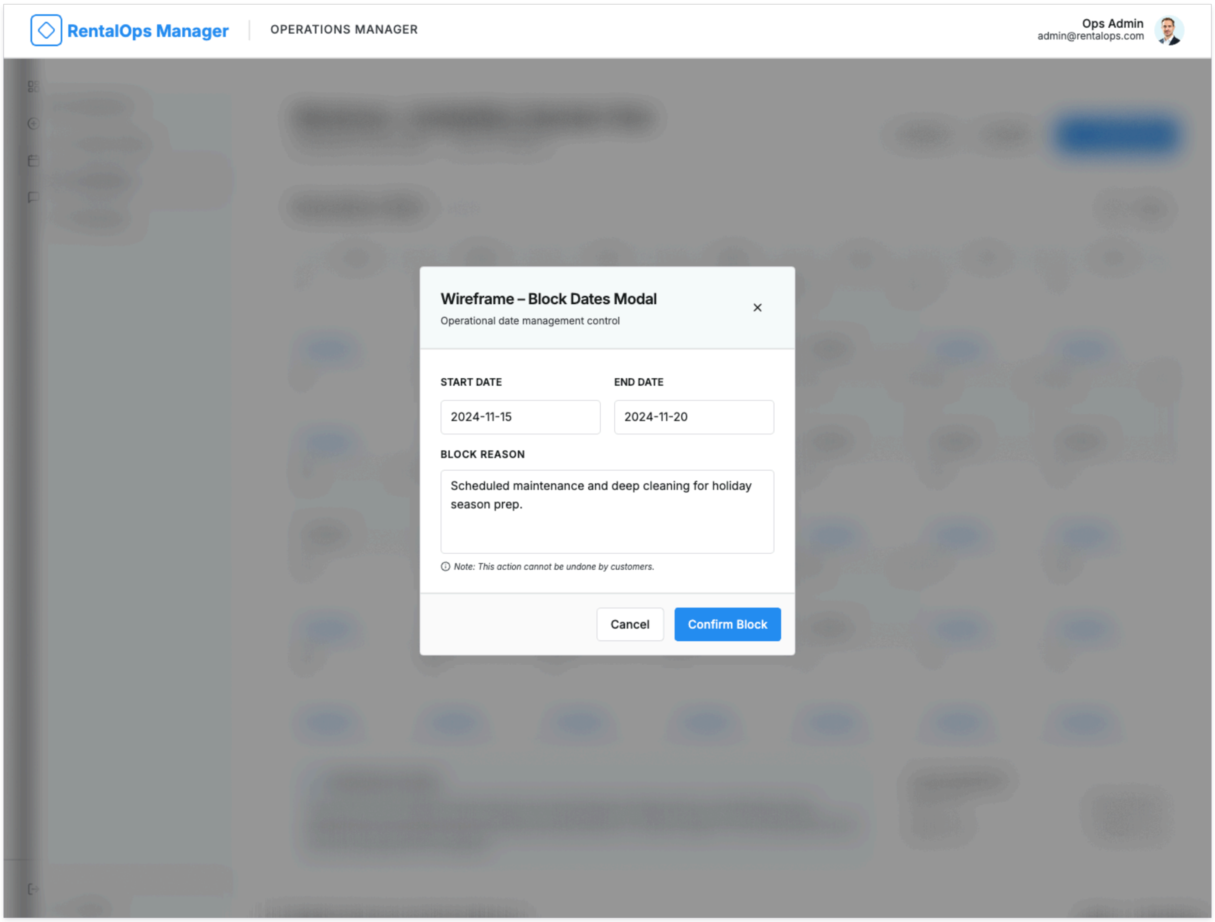
B. Create Listing



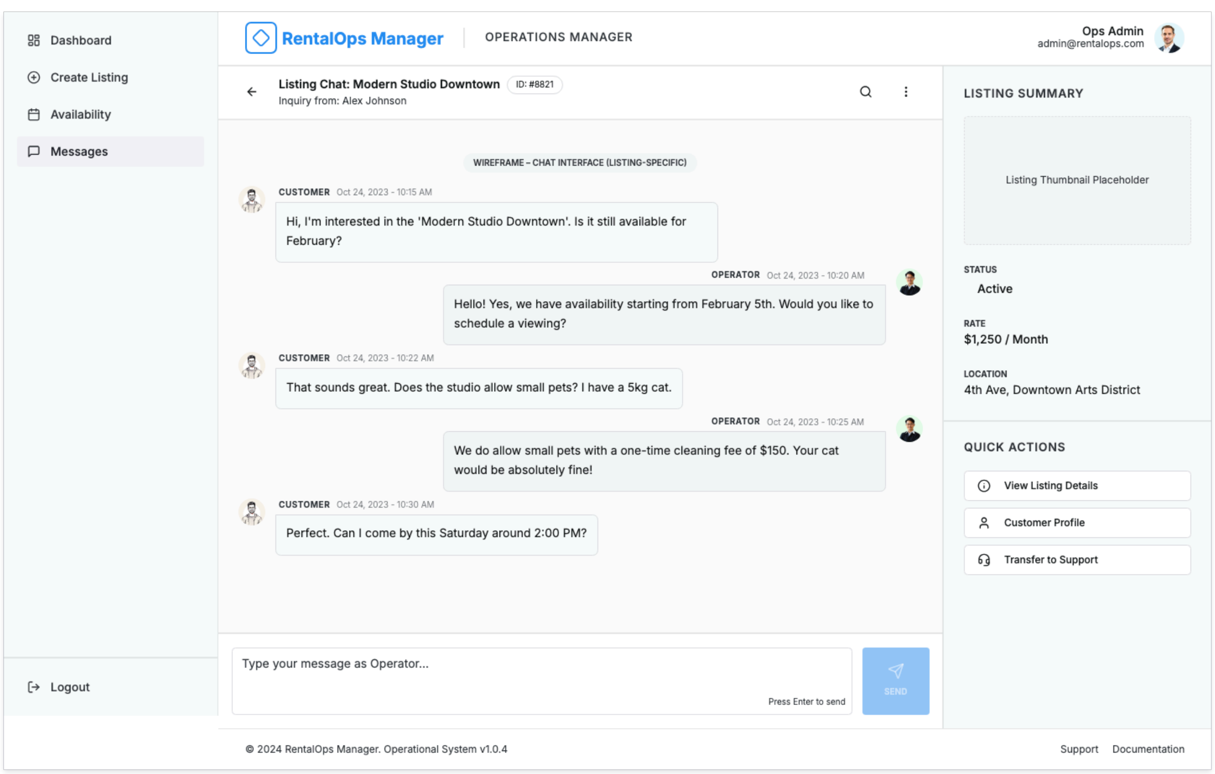
C. Listing Detail



D. Availability Calender



E. Listing Chat



F. Customer Listing View

RentalOps Manager | CUSTOMER VIEW | Login

WIREFRAME – CUSTOMER VIEW – LISTING PAGE • LISTING ID: #STUDIO-402

Modern Waterfront Studio - Unit 402

South Harbor, Marina District · Min. Stay: 2 Nights · Verified Listing

Experience upscale urban living in this sun-drenched studio overlooking the harbor. Features modern appliances, premium finishes, and high-speed internet. Perfect for business travelers or weekend getaways.

BOOKING INFO

$145 / night

Pricing is seasonal and may vary based on exact dates. Check availability below.

Contact Operator

Requests usually answered within an hour

Availability: June 2024

Available · Blocked

SUN	MON	TUE	WED	THU	FRI	SAT
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	31	1	2	3	4

Important Note

This calendar is for viewing purposes only. Dates marked as "Blocked" are already reserved or scheduled for maintenance. Please contact the operator to confirm availability for specific date ranges or to request a direct booking link.

AMENITIES

- High-speed Wi-Fi
- Full Kitchenette
- Smart TV (Streaming Ready)
- Workspace / Desk

HOUSE RULES

- No Smoking
- No Pets Allowed
- Check-in: 3:00 PM
- Check-out: 11:00 AM

LOCATION FEATURES

Located in the vibrant Marina District. Within walking distance to 5-star restaurants, the public ferry terminal, and boutique shopping galleries.

RentalOps

High-fidelity operations for rental professionals.

Product

Listings
Availability
Communication

Company

About Us
Terms
Privacy

Support

Help Center
Contact

9. Feature Breakdown

Frontend Responsibilities

- Listing creation forms
- Availability calendar UI
- Date selection and blocking UI
- Chat interface

Backend Responsibilities

- Listing APIs
- Availability validation and conflict detection
- State machine enforcement
- Chat message validation and storage

---

10. API Planning (High-Level)

This section outlines the key backend APIs required to support listing management, availability control, and customer–operator communication. All APIs follow REST principles and use JSON for request and response payloads. Backend services are responsible for validation, conflict detection, authorization, and data persistence.

10.1 Listing APIs

POST /listings

Purpose:

Create a new rental listing in the system.

Description:

This API accepts basic listing details and creates a new listing record in the database. Newly created listings are initialized in the **Draft** state. Further configuration (availability setup, validation) is required before activation.

Key backend responsibilities:

- Validate input fields
- Assign unique listing ID
- Set initial lifecycle state
- Persist listing data in PostgreSQL

GET /listings/:id

Purpose:

Fetch details of a specific rental listing.

Description:

Returns listing metadata, current lifecycle state, and readiness status. This API is used by the frontend to display listing information and determine allowed actions.

Key backend responsibilities:

- Validate listing ID
- Fetch listing data from database
- Return current state and configuration status

PATCH /listings/:id/state

Purpose:

Update the lifecycle state of a listing.

Description:

Allows controlled state transitions such as Draft → Active or Active → Paused. The backend enforces valid transitions using a state machine and checks listing readiness before activation.

Key backend responsibilities:

- Validate state transition
- Enforce lifecycle rules
- Update listing state atomically
- Generate operational alerts if needed

10.2 Availability APIs

POST /availability/block

Purpose:

Block specific dates or time ranges for a listing.

Description:

This API creates an availability block after verifying that the requested dates do not overlap with existing blocks. Both full-day and partial-day blocking are supported.

Key backend responsibilities:

- Validate date or time range
- Check listing state
- Detect overlapping availability blocks
- Insert block using transactional database writes

GET /availability/:listingId

Purpose:

Retrieve availability information for a listing.

Description:

Returns all availability blocks associated with a listing. Used by the calendar UI to display blocked and available dates.

Key backend responsibilities:

- Fetch availability blocks efficiently
- Apply date filters if provided
- Return data in calendar-friendly format

POST /availability/validate

Purpose:

Pre-validate availability without creating a block.

Description:

Used by the frontend to check whether a date range is available before submitting a block request. This API performs conflict detection but does not modify database state.

Key backend responsibilities:

- Run overlap detection logic
- Return validation result
- Avoid database writes

10.3 Chat APIs

POST /chat/message

Purpose:

Send a message related to a specific listing.

Description:

Allows customers and operators to communicate within a listing-specific chat context. Messages are validated and stored by the backend.

Key backend responsibilities:

- Validate sender role and access
- Ensure listing is eligible for chat
- Persist message in database
- Maintain message order and timestamps

GET /chat/:listingId

Purpose:

Retrieve chat messages for a listing.

Description:

Returns the message history for a listing. Used by the chat UI to display past communication between customers and operators.

Key backend responsibilities:

- Authorize chat access
- Fetch message history efficiently
- Support pagination if required

---

11. Database Entity Identification

This section defines the core database entities used in the system along with their attributes, data types, constraints, and purpose.

11.1 Listing Entity

Field Name	Data Type	Constraints	Description

listing_id	UUID	Primary Key, Not Null	Unique identifier for each rental listing
title	VARCHAR(255)	Not Null	Title or name of the rental listing
status	ENUM	Not Null	Current lifecycle state (Draft, Active, Paused, Disabled)
created_at	TIMESTAMP	Not Null, Default NOW()	Timestamp when the listing was created

11.2 AvailabilityBlock Entity

Field Name	Data Type	Constraints	Description
block_id	UUID	Primary Key, Not Null	Unique identifier for the availability block
listing_id	UUID	Foreign Key, Not Null	References the associated listing
start_date	DATE / TIMESTAMP	Not Null	Start date or time of the blocked period
end_date	DATE / TIMESTAMP	Not Null	End date or time of the blocked period
block_reason	VARCHAR(255)	Nullable	Reason for blocking (maintenance, hold, etc.)

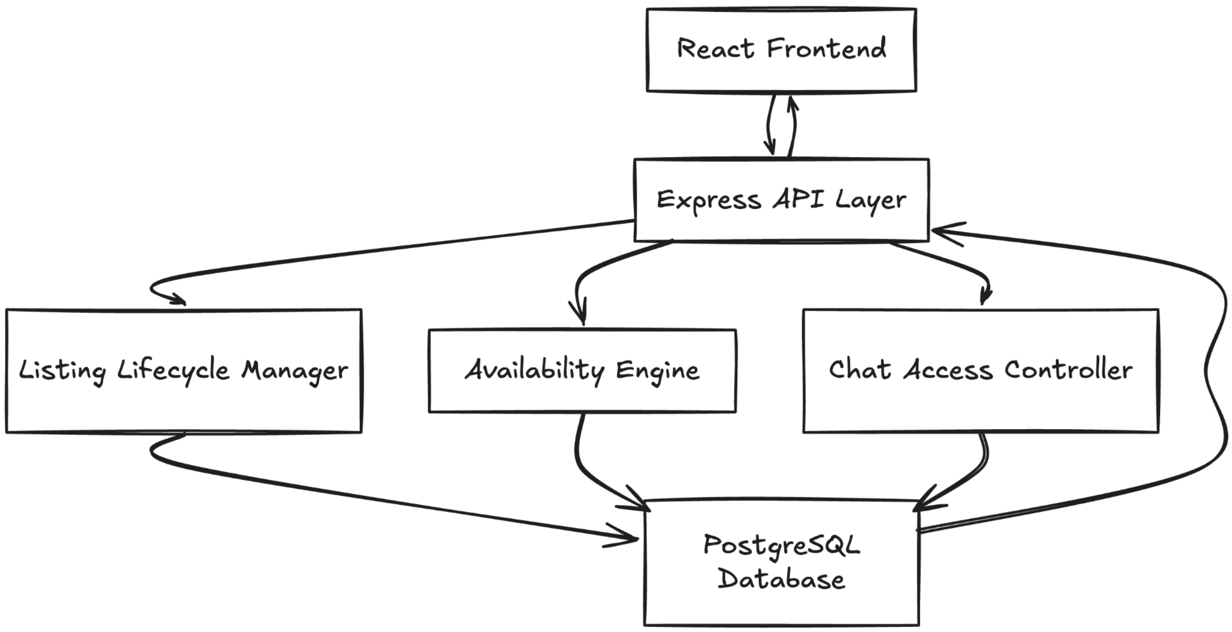
11.3 ChatMessage Entity

Field Name	Data Type	Constraints	Description
message_id	UUID	Primary Key, Not Null	Unique identifier for each chat message
listing_id	UUID	Foreign Key, Not Null	References the associated listing
sender_role	ENUM	Not Null	Sender role (Customer / Operator)
message_text	TEXT	Not Null	Content of the chat message
created_at	TIMESTAMP	Not Null, Default NOW()	Timestamp when the message was sent

Entity Relationships Summary

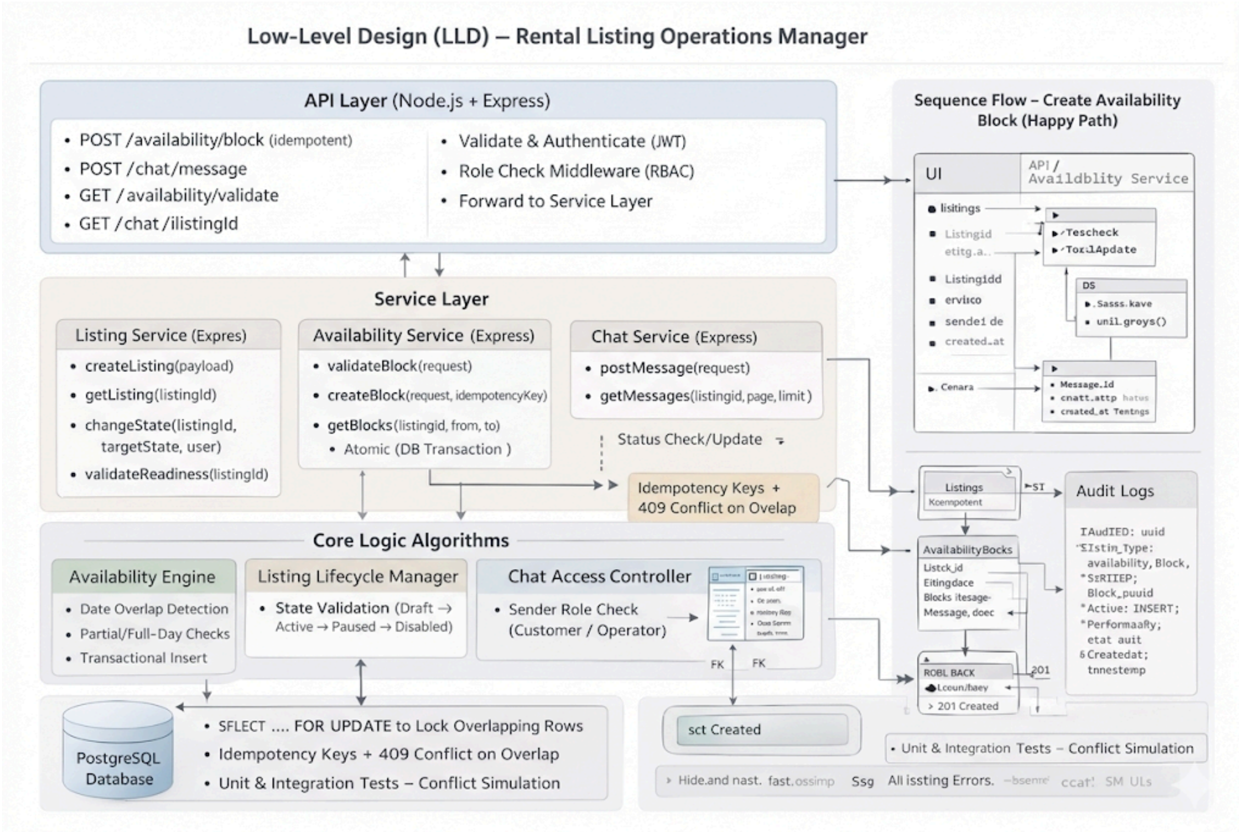
Relationship	Type
Listing → AvailabilityBlock	One-to-Many
Listing → ChatMessage	One-to-Many

12. High-Level Design (HLD) Diagram



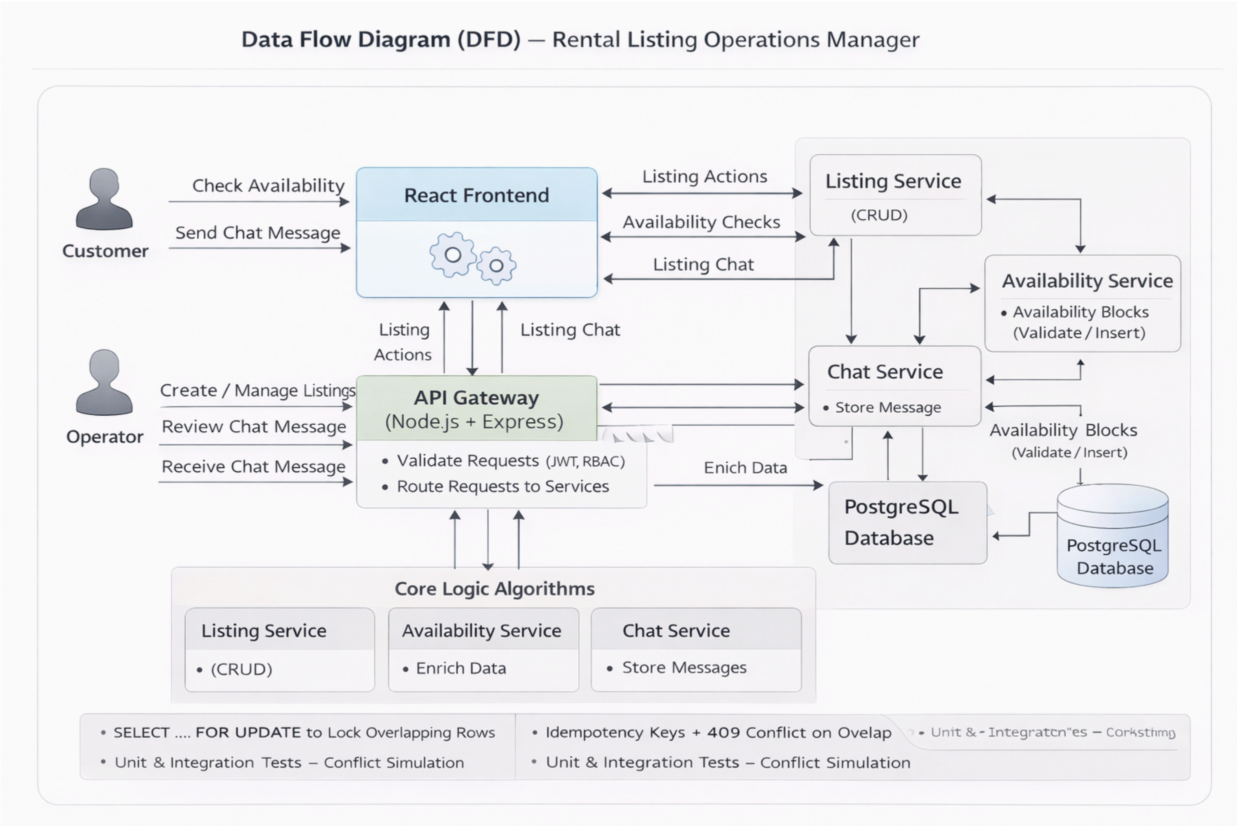
---

13. Low-Level Design (LLD) Diagram



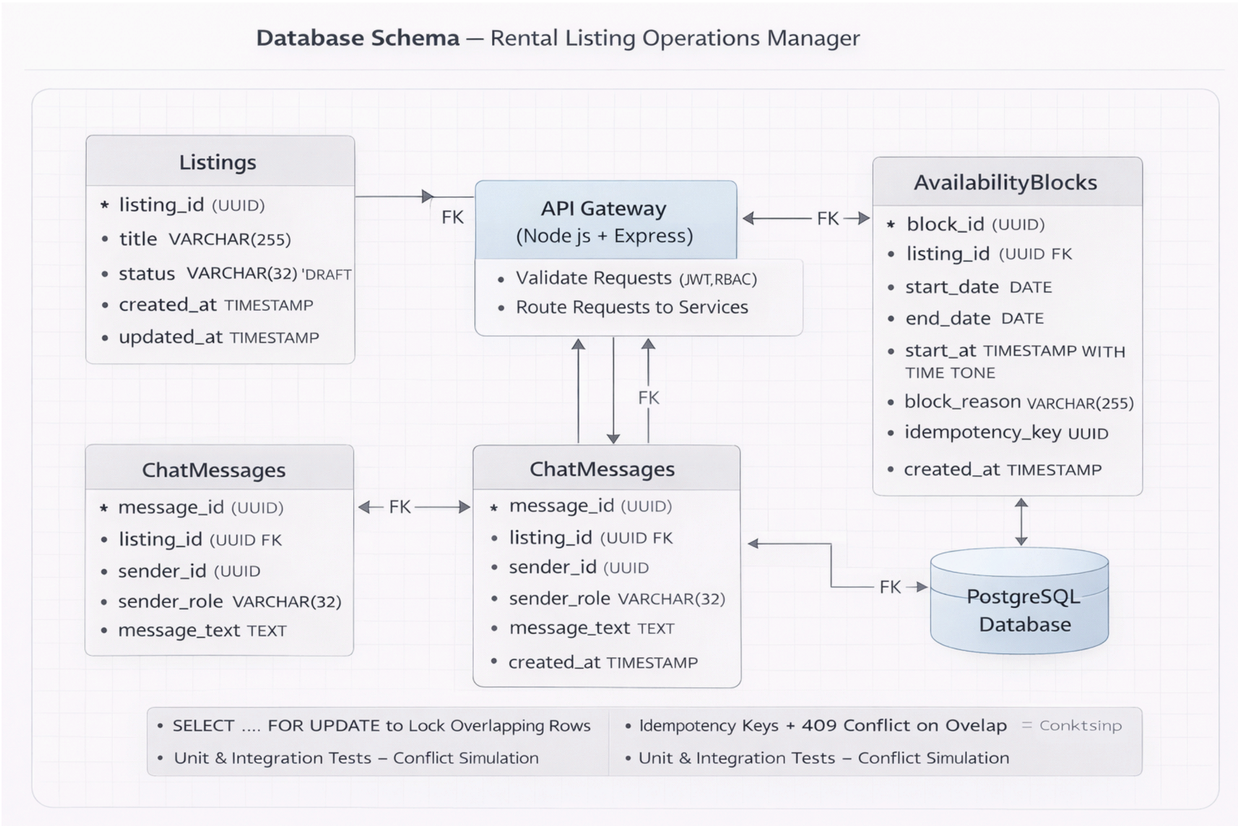
---

14. Data Flow Diagram (DFD)



Data Flow Diagram (DFD) — Rental Listing Operations Manager
Customer
Check Availability
Send Chat Message
React Frontend
Listing Actions
Availability Checks
Listing Chat
Listing Service
(CRUD)
Availability Service
• Availability Blocks (Validate / Insert)
Listing Actions
Listing Chat
Operator
Create / Manage Listings
Review Chat Message
Receive Chat Message
API Gateway (Node.js + Express)
• Validate Requests (JWT, RBAC)
• Route Requests to Services
Chat Service
• Store Message
Availability Blocks (Validate / Insert)
Enich Data
PostgreSQL Database
PostgreSQL Database
Core Logic Algorithms
Listing Service
• (CRUD)
Availability Service
• Enrich Data
Chat Service
• Store Messages
• SELECT .... FOR UPDATE to Lock Overlapping Rows
• Unit & Integration Tests – Conflict Simulation
• Idempotency Keys + 409 Conflict on Ovelap
• Unit & Integration Tests – Conflict Simulation

15. Database Schema Diagram



---

16. Non-Functional Requirements

- API response time < 500 ms
- System uptime > 95%
- Strong data integrity
- Secure backend endpoints
- Scalable architecture

17. Evaluation Readiness Summary

- Industry-relevant problem
- Backend-heavy logic
- Ops-grade system design
- Scalable architecture
- Real-world applicability

18. Project Timeline (12 Weeks)

Week	Phase	Objectives	Tasks & Deliverables
Week 1	Requirement Analysis	Understand problem & scope	Study rental platform workflows, finalize PRD structure, identify core features
Week 2	System Design	Architecture planning	Define HLD, identify system components, finalize tech stack
Week 3	Database Design	Data modeling	Design entities, relationships, indexes, and constraints
Week 4	API Design	Contract planning	Define listing, availability, and chat APIs (request/response, validations)
Week 5	Listing Module	Core listing logic	Implement listing creation, lifecycle states, readiness validation
Week 6	Availability Engine	Core business logic	Implement date overlap detection, block creation, conflict prevention
Week 7	Availability APIs	Backend integration	Build availability APIs, transaction handling, validation logic
Week 8	Chat Module	Communication system	Implement listing-specific chat APIs and database storage
Week 9	Frontend UI	User interaction	Build React UI for listings, calendar view, and chat interface

Week 10	Testing & Validation	Quality assurance	Unit tests, API testing, edge-case handling, concurrency testing
Week 11	Deployment	Production readiness	Deploy backend & frontend to cloud, configure environment
Week 12	Documentation & Review	Final submission	Prepare final PRD, diagrams, demo video, and project presentation